

Devoir 3 (H26)

MAT-2910 Analyse numérique pour l'ingénierie

Département de mathématiques et statistique de l'Université Laval

0 Consignes (Python)

- Vous pouvez former des équipes d'**au plus** 2 personnes. Il faut créer l'équipe sur monPortail avant la date limite indiquée (**même si on est seul!**).
- L'équipe doit être différente de celle du précédent devoir, sauf si vous êtes seul(e).
- Vous n'avez pas droit à l'aide du CDA pour ces exercices. Vous pouvez utiliser le forum, demander à l'auxiliaire du cours ou à l'enseignant.
- Dans la boîte de dépôt de monportail, vous devez remettre un seul fichier : un dossier compressé **en format .zip** par équipe nommé EquipeX.zip, où X est votre numéro d'équipe. Ce dossier devra contenir un **rapport écrit au traitement de texte** (*e.g.* MS Word, L^AT_EX) **converti en format .pdf** contenant tout ce qui est demandé pour les exercices (calculs, figures, explications, valeurs numériques, etc.). Le dossier contiendra aussi les codes **au format .py** demandés.
- **Codes demandés :** Le jeu de données fourni `points.txt` et six (6) programmes Matlab : un script `equipeXX.py` où XX désigne le numéro de votre équipe, les deux fonctions fournies `reglinA/B.py` (**ne pas les modifier**), la fonction `newton.py` complétée, et vos deux fonctions `regfreqA/B.py`.
- Le script doit générer toutes les figures demandées lorsqu'on l'exécute, et elles doivent concorder avec celles dans votre rapport. Dans le cas contraire, le code se verra attribuer la note de 0.
- Des points sont attribués à la présentation des figures (propreté, identification des axes, titres, légendes, couleurs). N'oubliez donc pas d'en tenir compte lorsque vous les incluez au rapport.
- Votre travail doit être remis sur la boîte de dépôt monPortail avant la date limite indiquée sur le site de cours. En cas de retard, une pénalité de 5%/heure s'applique

lors des premières 24h, jusqu'à concurrence de 20%. Après la première journée de retard, 20%/jour supplémentaire sera enlevé.

1 Importations

Dans votre script `equipeXX.py`, vous devrez inclure les importations suivantes :

```
import numpy as np
import matplotlib.pyplot as plt
from time import time as time
from reglinA import reglinA
from reglinB import reglinB
from newton import newton
```

et après les avoir codé, ajouter :

```
from regfreqA import regfreqA
from regfreqB import regfreqB
```

2 Questions

Soit un ensemble de n points donnés de $\mathbb{R}^2$, $X = \{(x_i, y_i) : i = 1, \dots, n\}$. On cherchera la meilleure courbe $y = f(x)$ qui représentera les points ; le problème est donc de trouver une fonction f telle que $f(x_i) \approx y_i$ pour chaque paire (x_i, y_i) . On appelle ce problème une **régression**. Une fois la fonction f trouvée, on peut s'en servir pour prédire une valeur de y pour un nouveau point x ne se trouvant pas dans le jeu de données initial.

2.1 Régression linéaire

Afin de simplifier la tâche, on peut tracer le graphique des données X pour deviner à l'avance une forme particulière pour f . Par exemple, si on voit une tendance linéaire dans les points, on peut supposer que $f(x) = \beta_1 + \beta_2 x$, où β_1 et β_2 seront des coefficients à déterminer (on parle dans ce cas de **régression linéaire**).

Pour déterminer les paramètres, on doit construire un système linéaire. Dans un monde idéal, on voudrait

$$f(x_i) = y_i \iff \beta_1 + \beta_2 x_i = y_i$$

pour chaque $i = 1, \dots, n$. On a donc n équations linéaires dont les inconnues sont β_1, β_2 ,

$$\begin{aligned}\beta_1 + \beta_2 x_1 &= y_1 \\ \beta_1 + \beta_2 x_2 &= y_2 \\ &\vdots \\ \beta_1 + \beta_2 x_n &= y_n\end{aligned}$$

qu'on peut exprimer sous forme matricielle $A\vec{\beta} = \vec{y}$. La matrice A est de dimension $n \times 2$:

$$A = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}.$$

Cette matrice étant rectangulaire, elle n'est donc **pas inversible** (il n'y a **pas de solution** au système). On définit le résidu comme un vecteur $F(\vec{\beta})$ ayant pour composantes

$$(F(\vec{\beta}))_i = f(x_i) - y_i.$$

En particulier, dans le cas où f est linéaire, $F(\vec{\beta}) = A\vec{\beta} - \vec{y}$. On voudrait trouver le vecteur de coefficients $\vec{\beta}$ qui rend $\|F(\vec{\beta})\|$ le plus petit possible, c'est-à-dire

$$F(\vec{\beta}) = \min_{z \in \mathbb{R}^2} \|F(z)\|,$$

qu'on appelle un *problème de moindres carrés*.

On propose deux approches pour résoudre ce problème, en modifiant le système pour qu'il admette une solution.

(A) On va créer un système auxiliaire en multipliant par $A^\top$ des deux côtés :

$$A^\top A \vec{\beta} = A^\top \vec{y}.$$

La matrice $A^\top A$ étant carrée et inversible, on peut résoudre ce système. On vous donnera un code `reglinA.py` qui implémente cette approche, en calculant la décomposition LU de $A^\top A$.

(B) On va utiliser une factorisation QR de A , c'est-à-dire

$$A = QR,$$

où Q est $n \times 2$ et orthogonale ($Q^\top Q = I$) et R est 2×2 et triangulaire supérieure. En multipliant le système original par $Q^\top$ des deux côtés,

$$Q^\top A \vec{\beta} = Q^\top \vec{y} \implies Q^\top Q R \vec{\beta} = Q^\top \vec{y} \implies R \vec{\beta} = Q^\top \vec{y}.$$

On peut résoudre ce dernier système très facilement. On vous donnera un code `reglinB.py` qui implémentera cette approche.

- a) Faire tracer le graphique des points donnés dans le fichier `points.txt` (Figure 1). Pour ce faire, vous devrez utiliser la commande `X = np.loadtxt("points.txt")` pour charger le jeu de données et la commande `plt.scatter(X[:,0],X[:,1],s=3)` pour tracer le nuage de points (contenant $n = 1000$ points). Dans le rapport, commenter sur la validité de l'hypothèse de linéarité de f .
- b) Dans le rapport, expliquer comment résoudre le système $R\vec{\beta} = Q^\top \vec{y}$ efficacement en vous basant sur la structure particulière de R et combien d'opérations cela coûte. De plus, expliquer pourquoi, dans l'approche (A), le coût du calcul de la décomposition LU de $A^\top A$ est négligeable par rapport au coût de la multiplication $A^\top A$. Conclure sur le coût asymptotique (quand n devient très grand) de l'approche (A).
- c) Dans votre script, utiliser `reglinA` et `reglinB` pour obtenir des valeurs de β_1 et β_2 (les indiquer au rapport). Puis, sur un graphique, tracer le nuage de point et les 2 droites de régression données par chacune des méthodes sur l'intervalle $[1, 6]$, en utilisant la fonction `np.linspace(1,6)` (Figure 2). Commenter ce que vous observez. De plus, indiquer au rapport et donner un sens à la valeur $\|F(\vec{\beta})\|$ affichée par `reglinA/B`.

2.2 Régression non-linéaire

Supposons maintenant qu'avec davantage d'intuition, on remarque qu'un modèle en racine carrée pourrait bien représenter les points. On pose $f(x) = \beta_1 + \beta_2 \sqrt{x - \beta_3}$. Cependant, comme β_3 est à l'intérieur de la racine, on ne peut pas exprimer les équations à résoudre de manière linéaire comme avant. Il faut donc résoudre ce problème à l'aide la méthode de Newton.

- d) Dans le rapport, exprimer le système d'équations à résoudre dans le cas général, sous la forme d'un résidu $F(\vec{\beta}) = 0$ (s'inspirer de ce qui a été présenté dans le cas linéaire). Ensuite, exprimer la jacobienne du système en fonction de $\vec{\beta}$ et indiquer pourquoi on ne pourrait pas appliquer la méthode de Newton telle que vue en classe pour résoudre ce problème (justifier en se basant sur la forme de la jacobienne).
- e) En la modifiant la moins possible, utiliser une des approches (A) ou (B) pour compléter la fonction `newton.py` afin de pouvoir traiter le problème de régression (vous pouvez

copier et adapter la section “résolution” de `reglinA/B.py`). Expliquer le changement dans le rapport.

- f) Dans votre script, appeler la méthode de Newton modifiée avec comme point de départ $\vec{\beta} = (1, 1, 1)^\top$, la tolérance à 10^{-7} et $N_{max} = 20$, pour calculer les coefficients $\beta_1, \beta_2, \beta_3$ (les indiquer au rapport). Pour ce faire, vous devrez définir dans le script une fonction `F = lambda beta : ...` pour la fonction dont on cherche la racine et `J = lambda beta : ...` pour la matrice jacobienne, où `beta` est un vecteur ligne de taille 3. Puis, sur un même graphique, tracer la courbe de régression sur l’intervalle $[1, 6]$ et le nuage de point (Figure 3).
- g) Regarder les sorties d’erreurs et de résidus que la fonction `newton` vous affiche dans la console. Indiquer et interpréter dans le rapport la dernière valeur de $\|F(\beta^{(k)})\|$ et expliquer pourquoi elle ne tend pas vers 0. De plus utiliser les valeurs de la colonne $\|\delta\beta^{(k+1)}\| = \|\beta^{(k+1)} - \beta^{(k)}\|$ pour calculer l’ordre de convergence de la méthode dans le rapport. Ce que vous observez est-il en concordance avec la théorie ?

2.3 Régression dans l’espace des fréquences

La régression dans l’espace des fréquences se base sur un modèle de la forme

$$\begin{aligned} f(x) &= \beta_1 + \beta_2 \cos(x) + \beta_3 \cos(2x) + \cdots + \beta_k \cos((k-1)x) \\ &\quad + \beta_{k+1} \sin(x) + \beta_{k+2} \sin(2x) + \cdots + \beta_{2k-1} \sin((k-1)x) \\ &= \sum_{i=1}^k \beta_i \cos((i-1)x) + \sum_{i=k+1}^{2k-1} \beta_i \sin((i-k)x), \end{aligned}$$

pour une valeur de k fixée. L’avantage de ce type de régression est qu’elle peut représenter une courbe de nature presque arbitraire, pas seulement linéaire ou racine carrée, car elle repose sur la série de Fourier.

- h) En s’inspirant de ce qui a été fait dans le cas linéaire (partir du fait qu’on voudrait $f(x_i) = y_i$ pour chaque i), on peut construire un système linéaire $A\vec{\beta} = \vec{y}$ pour trouver les poids $\vec{\beta}$. Dans le rapport, donner l’expression de la matrice A et ses dimensions, pour n et k arbitraires.
- i) Créer, dans un fichier à part, une nouvelle fonction `regfreqA(X,k)` qui retournera le vecteur de coefficients $\vec{\beta}$ pour la régression de Fourier, en copiant le code de `reglinA` (**seulement changer la création de la matrice A**). Faire la même chose avec `reglinB`.

- j) Obtenir les coefficients β_i à l'aide de vos deux fonctions `regfreqA` et `regfreqB` en fixant $k = 5$ puis $k = 15$. Tracer les 4 courbes $f(x)$ correspondantes sur le même graphique sur l'intervalle $[1, 6]$ avec le nuage de point (Figure 4). De plus, indiquer au rapport et interpréter la valeur $\|F(\vec{\beta})\|$ affichée par `regfreqA/B` pour chacune des valeurs de k . Selon cette valeur, commenter sur la précision cette méthode de régression par rapport aux deux autres.
- k) En utilisant la commande `time()`, calculer combien de temps prend chacune des fonctions `regfreqA/B` pour calculer les coefficients, pour $k = 5$ et $k = 15$, et indiquer les au rapport (faire une moyenne sur 1000 exécutions). Indiquer aussi au rapport le conditionnement des matrices $A^\top A$ et R (pour $k = 5$ et $k = 15$) à l'aide de la commande `np.linalg.cond` (vous pouvez modifier les fonctions `regfreqA/B` pour faire afficher le conditionnement dans la console). Conclure sur l'efficacité et la fiabilité globale de chacune des approches selon k .
- l) Supposons que l'erreur relative causée par la représentation flottante des nombres sur $\vec{y}$ est d'environ 10^{-15} . Dans le rapport, calculer une borne supérieure à l'erreur relative commise sur $\vec{\beta}$ pour `regfreqA/B` (en prenant $k = 5$, puis $k = 15$), en justifiant avec la théorie vue en classe. Conclure sur la précision des deux approches selon k .